Politecnico di Torino

Politecnico di Torino

Lab report for the course Qubit Electronics

Master degree in Quantum Engineering

Authors: Group 3

Riola Leonardo Antonio, Donati Davide, Girotto Giacomo

May 15, 2026

Contents

1. Introduction

Giacomo sei un pollo
scemo chi legge

2. Changes in python script

In order to solve the Schrodinger equation for the device analyzed in this laboratory, we modified the given code whose initial aim was uniquely to calculate the potential inside the device throught the non-linear Poisson equation.

```
# Schrodinger configuration
    num_states = 5
    params_schrod = SchrodingerSolverParams()
    params_schrod.num_states = num_states
    params_schrod.tol = 1e-9
```

Listing 2.1: Initialization of Schrodinger parameters

First (2.1), we create the object `SchrodingerSolverParams()`, a class that will contain all the parameters that we want to set. Then, we defined the number of energy levels to consider to be equal to 5: this will tell to the program to consider only the first 5 solutions of the Schrodinger equation, storing the eigenfunctions with their correspective eigenvalues. Another paramater that is set is the threshold in the convergence process during the calculation: this value, which in this case is set to be equale to $1e-9$, indicates the degree of convergence we want to achieve in defining a solution valid: if the difference between two subsequential iteration for the same value is less than this threshold, then the value will be considered converged and so valid.
Then, after defining the parameters for the solver, the following commands are added:

```
    d.set_V_from_phi()
    submesh = SubMesh(d.mesh, dot_region)
    subd = SubDevice(d,submesh)

    schrod_solver = SchrodingerSolver(subd, solver_params=params_schrod)
    schrod_solver.solve()
```

Listing 2.2: Physical definition of the domain

- `d.set_V_from_phi()`: This command updates the internal state of the `Device` object `d` by populating a new attribute, the potential energy array `V`. Physically, it performs a node-by-node mapping of the electrostatic potential ϕ into the potential energy V using the relation $V(\mathbf{r}) = -q \cdot \phi(\mathbf{r})$. Without this step, the Schrödinger solver would lack the necessary confinement potential to find bound states.
- `SubMesh` and `SubDevice`: These commands initialize the phyisical domain where the Schrodinger equation will be calcualted. By defining a `submesh` limited to the `dot_region`, the simulation focuses

computational resources only on the volume where the quantum wavefunctions are expected to be non-zero. In fact, `dot_region` in an array defined previously in the code containing all the names of Phyisical Groups of our interest, which are:

```
dot_region=["region_pre_quantum_B_mid","region_quantum_B_mid",
            "region_post_quantum_B_mid"]
```

Listing 2.3: Regions containing the quantum dots

The `SubDevice` object `subd` inherits all physical properties (such as temperature and materials) from the main device but operates on this significantly smaller mesh, drastically reducing memory usage and computation time.

- `SchrodingerSolver` and `.solve()`: These commands initialize and execute the numerical solver for the time-independent Schrödinger equation. The solver transforms the physical Hamiltonian operator into a large, sparse matrix using the Finite Element Method (FEM), subsequently solving the resulting eigenvalue problem $H\psi = E\psi$.

After the calculation, the object `subd` will contain the first 5 solutions of the Schrodinger equation, that will subsequentially extracted and used to generate the output file `DQD_SiMOS_q.vtu` and `DQD_SiMOS_energies.txt`. The first is used in Paraview in order to visualize the shape and value of the wave functions, while in the second text file are reported the eigenvalues (energy values in eV) of the relative eigenfunction.